



EV ADAS Dashboard System

Real-Time Electric Vehicle Monitoring and Advanced Driver Assistance System

Presented by S. Shyam Kumar — Dept. of Electronics & Communication Engineering, Lendi Institute of Engineering and Technology

- 📄 Speaker notes: Introduce yourself and outline the project: an STM32-based simulated EV with ADAS, UART telemetry, and a Python Matplotlib dashboard for real-time monitoring and safety alerts.



Why Modern EVs Need Smart Dashboards

Modern electric vehicles produce continuous high-frequency telemetry across powertrain and sensors. Engineers and drivers require a consolidated, low-latency view to detect hazards and manage system health.

- Vehicle Speed
- Battery State of Charge (SOC)
- Motor Temperature & Torque
- Driving Range
- Obstacle Distance (ultrasonics)

Our solution: a real-time EV dashboard integrated with ADAS safety features and prioritized alarm management.

- Speaker notes: Emphasize the operational risk of fragmented telemetry and the benefits of centralized, prioritized alerts for safety and diagnostics.



Sensor Input

STM32 Blue Pill

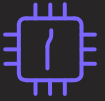
EV Controller

ADAS & Faults

Caption: End-to-end data flow from sensor acquisition on STM32, local ADAS processing and fault management, to UART telemetry and Python visualization.

- ❏ Speaker notes: Walk through each block: analog inputs (potentiometers/ultrasonics), STM32 ADC & control loop, ADAS modules, UART framing, VSPE virtual COM, Tera Term monitoring, and Python/Matplotlib dashboard rendering.

Development Tools & Why They Matter



STM32CubeIDE

Firmware development, debugging, and HAL integration for STM32F103C8T6.



PICSimLab

Hardware-in-the-loop simulation for sensors and peripheral timing validation.



Tera Term & VSPE

Serial monitoring and virtual COM port setup for telemetry inspection.



Python + Matplotlib

Real-time parser and plotting engine for dashboard visualization and ADAS overlays.



PySerial

Robust serial I/O layer for parsing framed telemetry at 115200 bps.

- ❏ Speaker notes: Briefly justify tool choices: deterministic firmware stack, accurate simulation, reliable serial bridging, and flexible plotting for rapid UI iteration.



Read Sensors

ADC Conversion

EV Dynamics

Compute Outputs

Caption: Execution cycle showing sensing, local estimation, safety checks, telemetry framing, and dashboard rendering.

- 📝 Speaker notes: Explain sampling cadence, ADC scaling, basic EV dynamics used for torque/range estimation, ultrasonic polling, and where latency/buffer management is handled.

Key Features Implemented



Real-Time Speed Monitoring

Instantaneous vehicle speed derived from encoder/potentiometer inputs and filtered via moving-average.



Battery SOC & Range Estimation

Voltage-based SOC estimator with simple coulomb-counting correction for short runs and range projection.



Motor Temperature & Torque

Thermal monitoring and torque calculation from control inputs and measured RPM proxies.



Ultrasonic-based ADAS

Forward collision, blind spot, parking assist, and time-to-collision (TTC) computations using distance and speed.



UART Shell & Fault Management

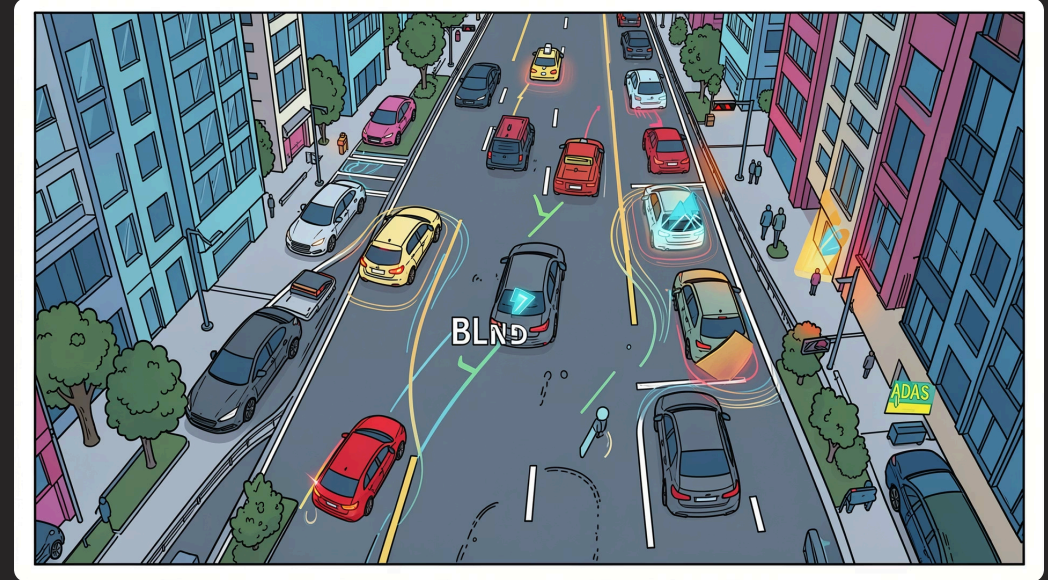
Interactive UART commands for diagnostics, plus prioritized alarm handling (Advisory → Warning → Critical).

📄 Speaker notes: For each feature, mention the implemented algorithmic approach and the telemetry fields sent over UART for visualization and logging.

ADAS Safety System

Functional Highlights

- **Forward Collision Warning** — distance threshold + closing velocity to trigger advisory/warning.
- **Blind Spot Detection** — side ultrasonic fusion with hysteresis to avoid false positives.
- **Time-To-Collision (TTC)** — $TTC = \text{distance} / \text{relative speed}$ (computed on STM32).
- **Parking Assist** — low-speed distance bands with graduated alarms and visual guidance.
- **Alarm Priority Levels** — None → Advisory → Warning → Critical (preemptive suppression rules).



- ❏ Speaker notes: Describe thresholds, how TTC is computed on-device, debouncing/hysteresis strategies, and priority arbitration for simultaneous alarms.

Python Dashboard — UI & Telemetry

Dashboard panels update at the telemetry rate via PySerial: numeric gauges (speed, SOC, range, torque, motor temp), ADAS bird-eye view, alarm strip, and fault indicator.

Speedometer

Digital + analog gauge with smoothing and peak markers.

Battery SOC & Range

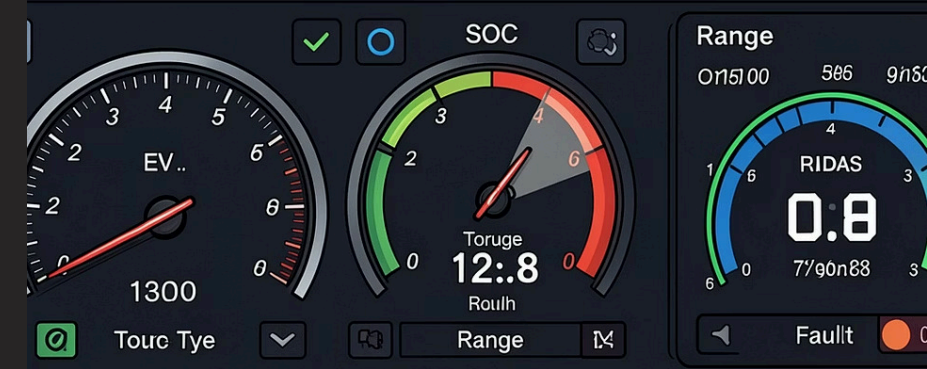
Live SOC %, projected range, and charge/consumption graph.

ADAS View

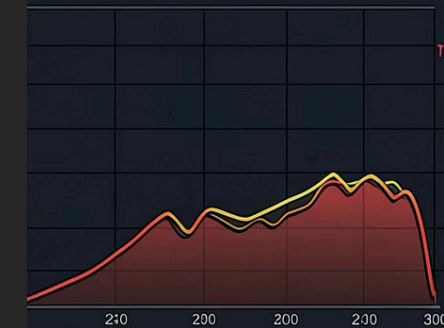
Top-down obstacle plotting, blind-spot flags, and TTC overlays.

- ❏ Speaker notes: Explain data parsing schema, frame structure, error checks (CRC/parity), and how Matplotlib subplots are organized for low-latency redraws.

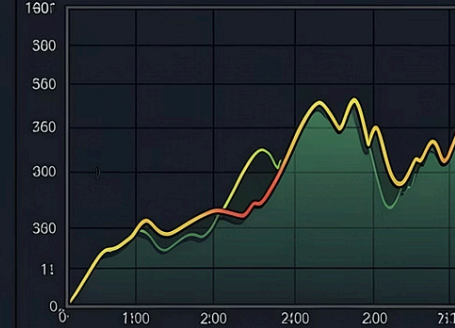
3, EV, Ggalour & ADAS



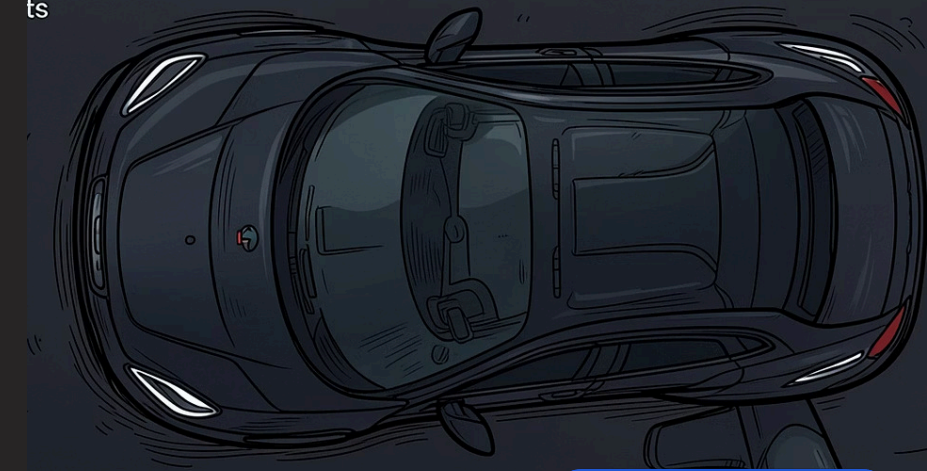
or Temp



Bard ADAS



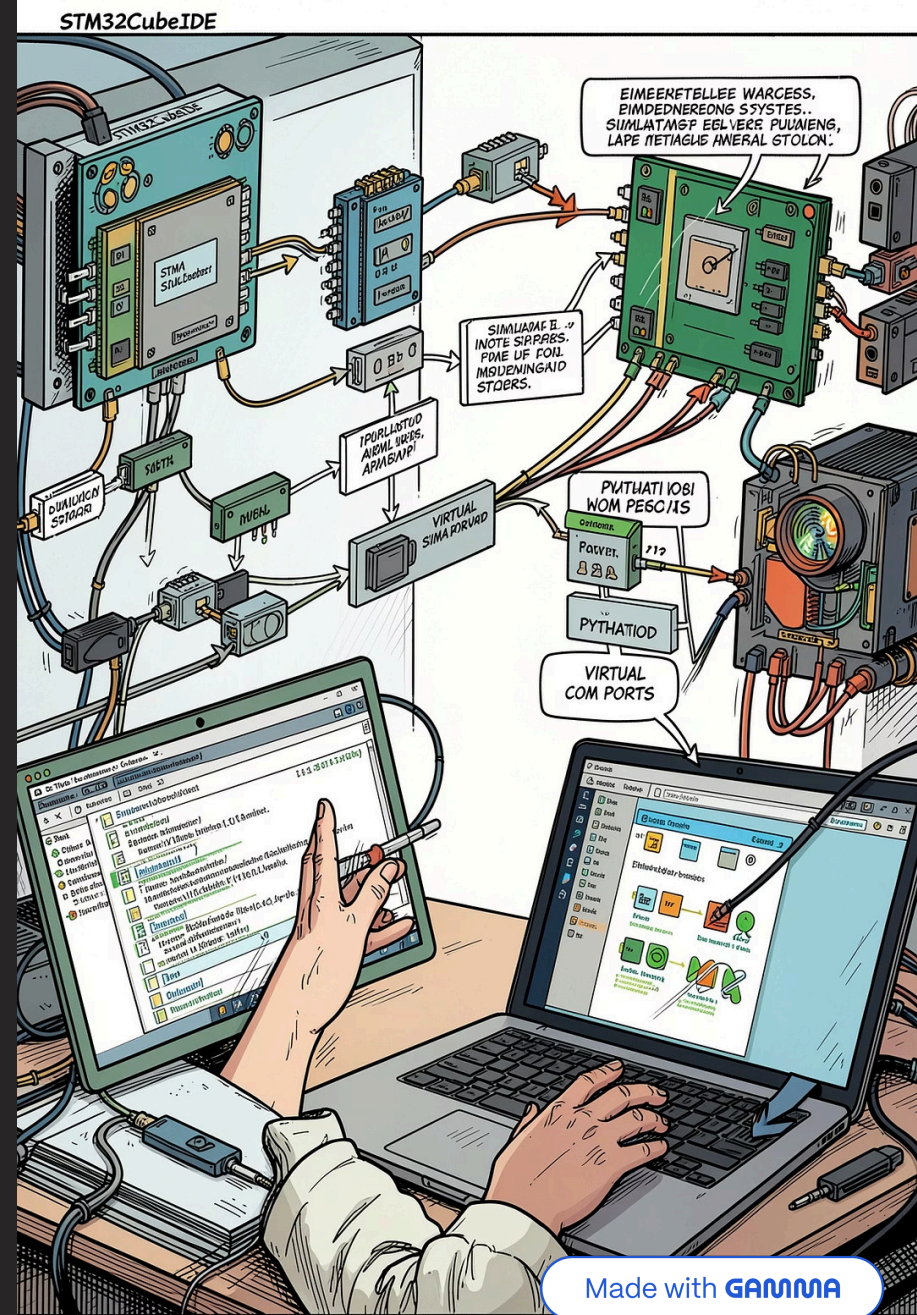
ts



Project Demonstration Sequence

1. Build firmware in STM32CubeIDE and flash Blue Pill.
2. Launch PICSimLab and initialize sensors (potentiometers, HC-SR04).
3. Configure VSPE to create virtual COM pair and open Tera Term to inspect UART.
4. Start Python dashboard (PySerial → parse → Matplotlib render).
5. Operate potentiometers and move simulated obstacles to observe speed, SOC, collision warnings, and blind-spot alerts.
6. Inject faults via UART shell and demonstrate alarm priority & reset flow.

- Speaker notes: Provide exact demo steps, estimated timings, common pitfalls (COM port conflicts, baud mismatch), and how to interpret dashboard signals during live run.



Results & Learning Outcomes

- Delivered a validated real-time EV monitoring stack with ADAS features implemented on STM32 and simulated in PICSimLab.
- Bi-directional UART telemetry (115200 bps) and Python/Matplotlib dashboard for live visualization and diagnostics.
- Implemented fault detection, alarm priority, TTC, blind-spot and parking assist functionalities.
- Skills gained: Embedded C, STM32 HAL, simulation-based testing, serial protocols, Python plotting, and ADAS engineering principles.

Thank you — Questions?

- 📄 Speaker notes: Conclude by summarizing technical achievements, potential next steps (sensor fusion with CAN bus, CAN FD integration, ML-based obstacle classification, hardware-in-the-loop on real ECU), and invite technical questions from the panel.

THANK YOU



Next Steps

CAN integration, higher-fidelity sensor models, and endurance testing.

Artifacts Included

Firmware repo, Python dashboard code, PICSimLab scenes, and demo script.